

Homework 05

```
In [1]: import numpy as np
import pandas as pd

import sklearn
import sklearn.model_selection
import sklearn.linear_model

import matplotlib.pyplot as plt

import pygam

import matplotlib

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

import scipy

matplotlib.rcParams['figure.figsize'] = (12,8)
```

Problem 1

```
In [2]: marketing = pd.read_csv("marketing.csv")
marketing.head()
```

```
Out[2]:
```

	age	job	marital	education	default	balance	housing	loan	y
0	58	management	married	tertiary	no	2143	yes	no	no
1	44	technician	single	secondary	no	29	yes	no	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	no
3	47	blue-collar	married	unknown	no	1506	yes	no	no
4	33	unknown	single	unknown	no	1	no	no	no

```
In [3]: X = pd.get_dummies(marketing.iloc[:,0:8])
y = np.where(marketing['y']=='yes',1,0)
```

```
In [4]: X_train, X_test, y_train, y_test = sklearn.model_selection.train_test_split(
                                                X, y,
                                                test_size = 0.33,
                                                random_state = 1)
```

a)

Base success rate (proportion of successful sales pitches from the calling campaign).

```
In [5]: base_rate = np.mean(y_train)
base_rate
```

```
Out[5]: 0.11782377603908752
```

b)

```
In [6]: from sklearn.linear_model import LogisticRegression
```

```
In [7]: log_reg = LogisticRegression(penalty='none')
log_reg.fit(X_train,y_train);

y_test_pred = log_reg.predict(X_test)
y_test_pred_proba = log_reg.predict_proba(X_test)[:,-1]
```

Looking at our training set we get a misclassification rate of:

```
In [8]: logistic_misclass_rate = np.mean(y_test_pred != y_test)
logistic_misclass_rate
```

```
Out[8]: 0.11548257372654155
```

c)

Just predicting everything as 0 (failure), gets us a misclassification rate of:

```
In [9]: np.mean(y_test != 0)
```

```
Out[9]: 0.11528150134048257
```

You should notice that this is basically the same as the missclassification rate as the logistic model.

d)

Top 1000 most likely clients...

```
In [10]: test_prob = log_reg.predict_proba(X_test)[:,-1]

top1000_id = np.argsort(test_prob)[:,-1][:1000]

logistic_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id])
logistic_success_in_top_1000
```

```
Out[10]: 0.291
```

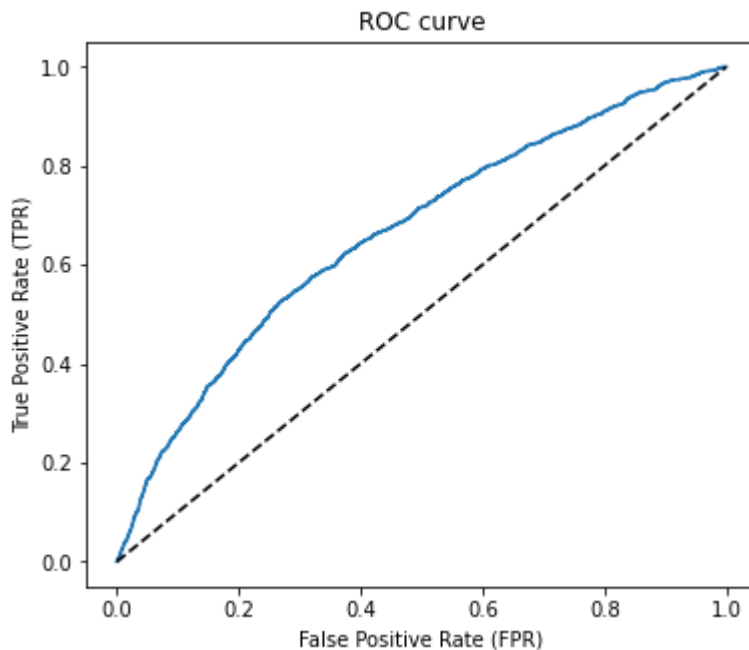
If we were to call the top 1000 most-likely-to-buy clients we would have obtained a success rate of 29%, whereas we would only expect 11.5% if we had randomly called individuals.

e)

```
In [11]: from sklearn.metrics import roc_curve
```

```
In [12]: fpr, tpr, _ = roc_curve(y_test, test_prob)
```

```
In [13]: fig, ax = plt.subplots(figsize = (6,5))
ax.plot(fpr, tpr)
ax.plot([0,1],[0,1],"k--")
ax.set(xlabel = "False Positive Rate (FPR)",
       ylabel = "True Positive Rate (TPR)",
       title = "ROC curve");
```



Problem 2

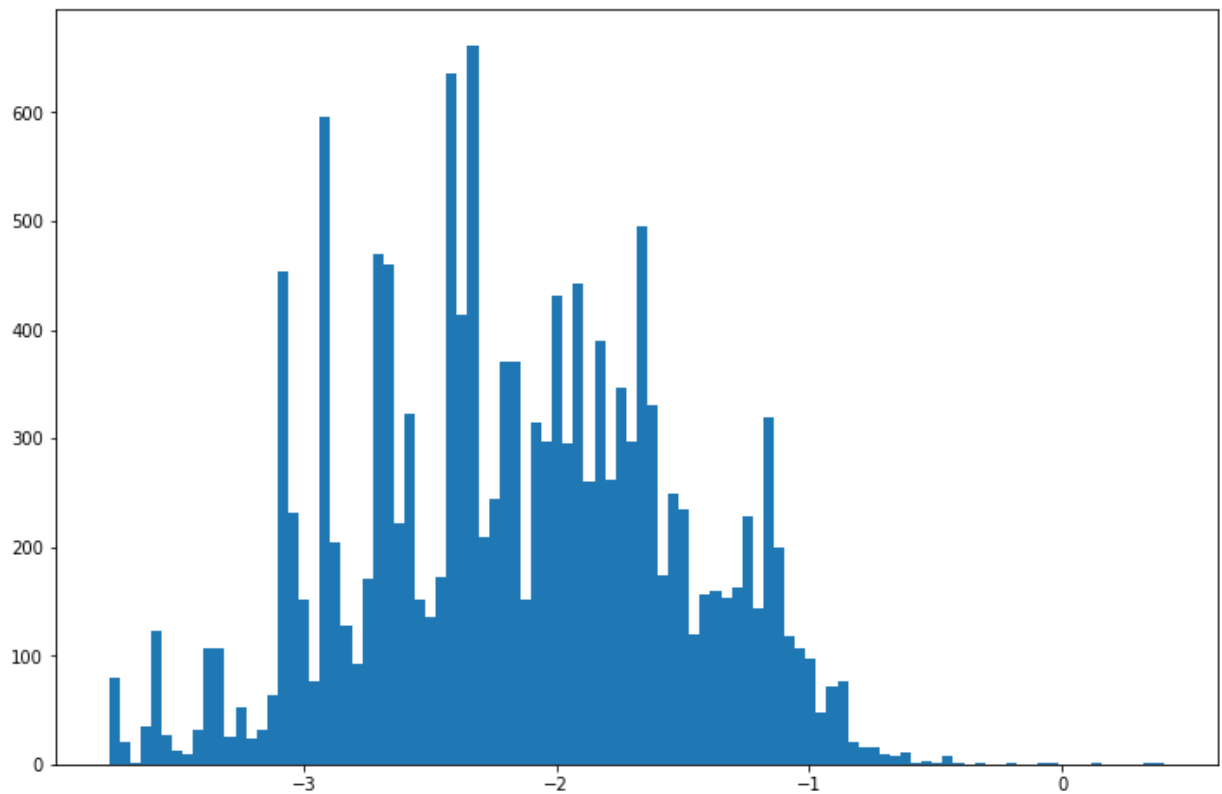
```
In [14]: y_test_pred = log_reg.predict(X_test)
y_test_pred_proba = log_reg.predict_proba(X_test)[:,1]
```

```
In [15]: def logit(p):
         return(np.log(p/(1 - p)))
```

a)

```
In [16]: test_prob_logit = logit(y_test_pred_proba)
```

```
In [17]: fig, ax = plt.subplots()
ax.hist(test_prob_logit, bins = 100);
```

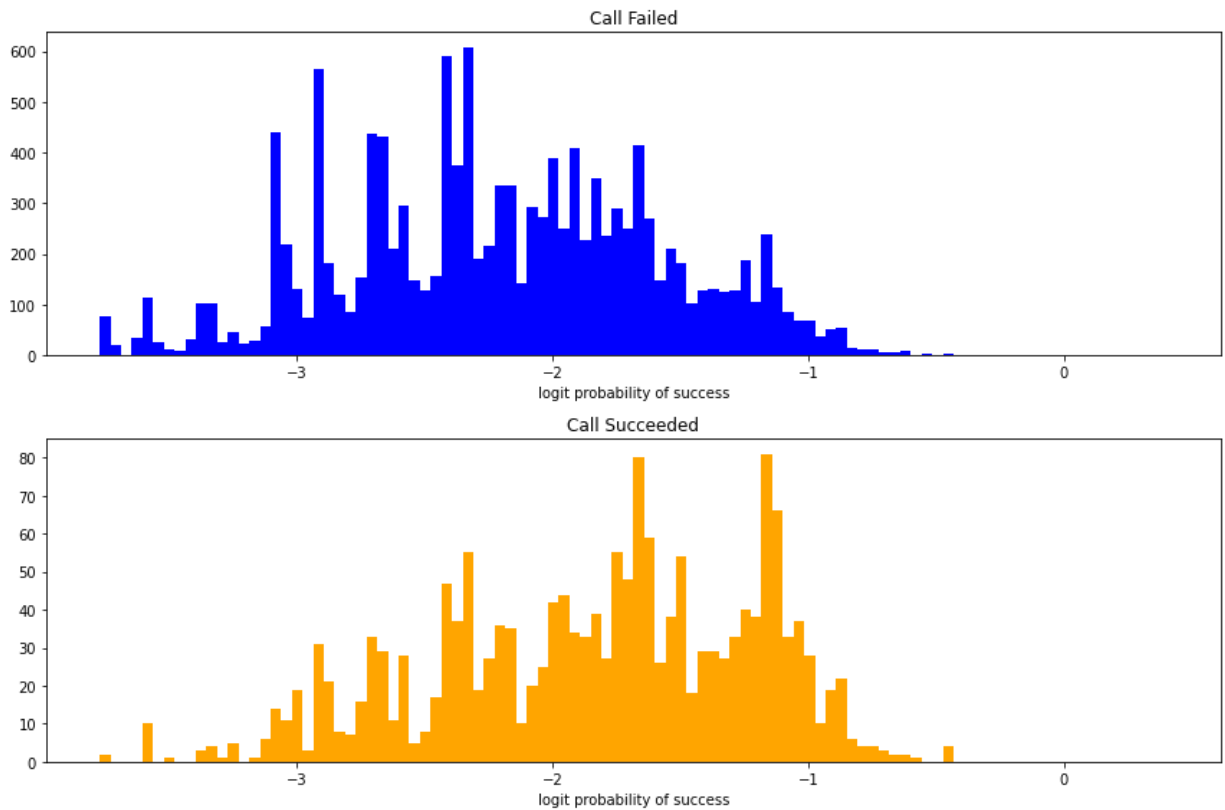


b)

```
In [18]: x_range = [np.min(test_prob_logit), np.max(test_prob_logit)]
         colors = ["blue", "orange"]
         titles = ["Call Failed", "Call Succeeded"]

         fig, axs = plt.subplots(nrows = 2)
         for i, ax in enumerate(axs):
             ax.hist(test_prob_logit[y_test == i],
                     bins = 100,
                     range = x_range,
                     color = colors[i]);
             ax.set(title = titles[i],
                    xlabel = "logit probability of success")

         fig.tight_layout()
```



c)

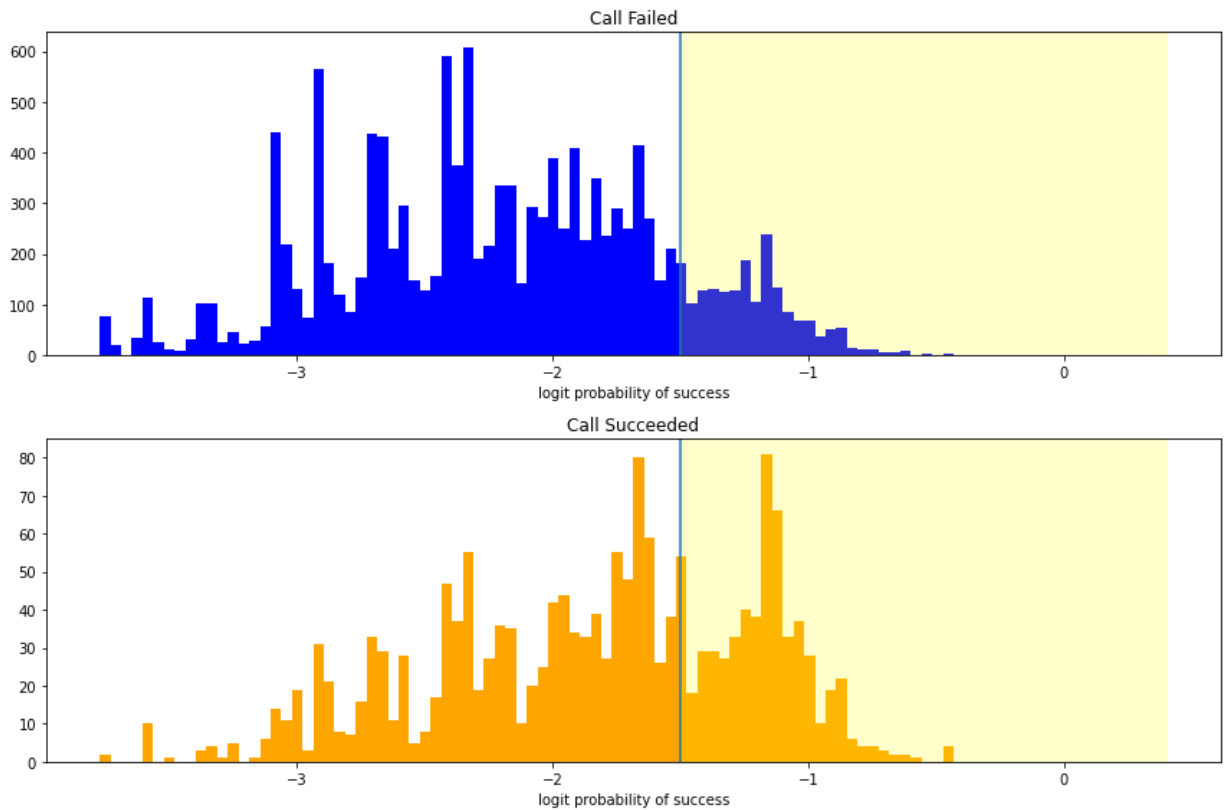
$$TPR = \frac{\#TP}{\#True} = \int_T^{\infty} f_1(x)dx$$

$$FPR = \frac{\#FP}{\#False} = \int_T^{\infty} f_0(x)dx$$

```
In [19]: x_range = [np.min(test_prob_logit), np.max(test_prob_logit)]
colors = ["blue", "orange"]
titles = ["Call Failed", "Call Succeeded"]

fig, axs = plt.subplots(nrows = 2)
for i, ax in enumerate(axs):
    ax.hist(test_prob_logit[y_test == i],
            bins = 100,
            range = x_range,
            color = colors[i]);
    ax.set(title = titles[i],
           xlabel = "logit probability of success")
    ax.axvline(-1.5)
    ax.axvspan(-1.5, x_range[1], alpha=0.2, color='yellow')

fig.tight_layout()
```



The shaded region above $T = -1.5$ in the first graphic ("Call Failed") corresponds to False Positive observations (observations that were labeled positive but were actually negative), and the proportion of observations in this graphic in this region is the FPR .

The shaded region above $T = -1.5$ in the second graphic ("Call Succeeded") corresponds to True Positive observations (observations that were labeled positive but were actually negative), and the proportion of observations in this graphic in this region is the TPR .

d)

AUC in terms of an integral involving TPR and FPR.

Below we use the parameterization of x and y by t to get that when we have $x = f(t), y = g(t)$, then the area under the curve is $\int_{t_0}^{t_f} g(t) f'(t) dt$.

$$\begin{aligned} y = h(T) &= \int_T^\infty f_1(\xi) d\xi \\ x = f(T) &= \int_T^\infty f_0(\xi) d\xi = 1 - \int_{-\infty}^T f_0(\xi) d\xi \end{aligned}$$

$$\begin{aligned} f'(T) &= -f_0(T) \\ AUC &= \int_{-\infty}^\infty -f_0(T) \left(\int_T^\infty f_1(x) dx \right) dT \\ &= \int_{-\infty}^\infty f_0(T) \left(\int_T^\infty f_1(x) dx \right) dT \end{aligned}$$

Which should be written:

$$AUC = \int_{-\infty}^\infty -FPR'(T) TPR(T) dT$$

e)

Associated $\mathbb{P}(S_1 > S_0)$ with AUC.

$$\begin{aligned} \mathbb{P}(S_1 > S_0) &= \int_{S_1} \int_{S_0} 1(S_1 > S_0) f_1(S_1) f_0(S_0) dS_0 dS_1 \\ &= \int_{S_0=-\infty}^\infty \int_{S_1>S_0} f_1(S_1) f_0(S_0) dS_0 dS_1 \\ &= \int_{-\infty}^\infty \int_T^\infty f_0(T) f_1(S_1) dS_1 dT && \text{(via change of variable)} \\ &= \int_{-\infty}^\infty f_0(T) \left(\int_T^\infty f_1(x) dx \right) dT \\ &= AUC \end{aligned}$$

Problem 3

```
In [20]: def eval_estimate(estimate, truth, loss_FP, loss_FN):
        """
        Calculates Sensitivity, specificity and Loss

        This function should take in numpy logical arrays for estimate and truth
        and single values for the losses
        """
        # note you need both estimate and truth to be boolean to use "~"
        estimate = np.array(estimate, dtype = bool)
        truth     = np.array(truth, dtype = bool)

        nP = np.sum(truth)
        nN = np.sum(~truth)
        FP = np.sum(estimate[~truth]) # OR np.sum((~truth) & (estimate))
        FN = np.sum(~estimate[truth]) # OR np.sum((truth) & (~estimate))
        loss = loss_FP * FP + loss_FN * FN

        sens = (nP - FN)/nP #TPR
        spec = (nN - FP)/nN #TNR

        return(sens, spec, loss)
```

```
In [21]: def logit(p):
        return(np.log(p/(1 - p)))
        test_prob = log_reg.predict_proba(X_test)[: ,1]
        test_prob_logit = logit(test_prob)
```

```
In [22]: threshold = -1
        loss_FP = 5
        loss_FN = 100
        eval_estimate(test_prob_logit > -1, y_test, 5, 100)
```

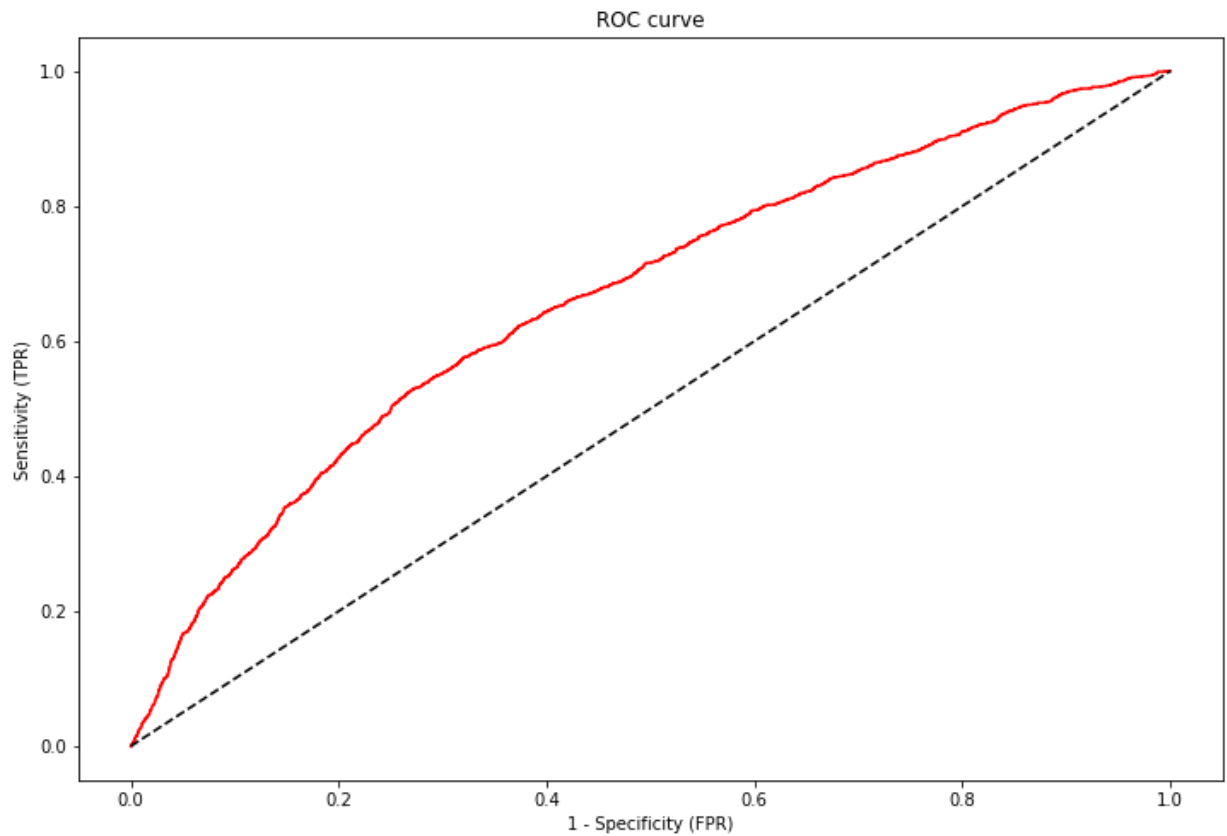
```
Out[22]: (0.053488372093023255, 0.9808333333333333, 164065)
```

b

Calculating ROC curve values

```
In [23]: unique_values = np.sort(np.unique(test_prob_logit))
        midpoints = (unique_values[0:(len(unique_values) - 1)] +\
                     unique_values[1:len(unique_values)] ) / 2.0
        sens, spec, loss = np.vectorize(
                        lambda x: eval_estimate(test_prob_logit > x,
                                                y_test, 5, 100) )(midpoints)
```

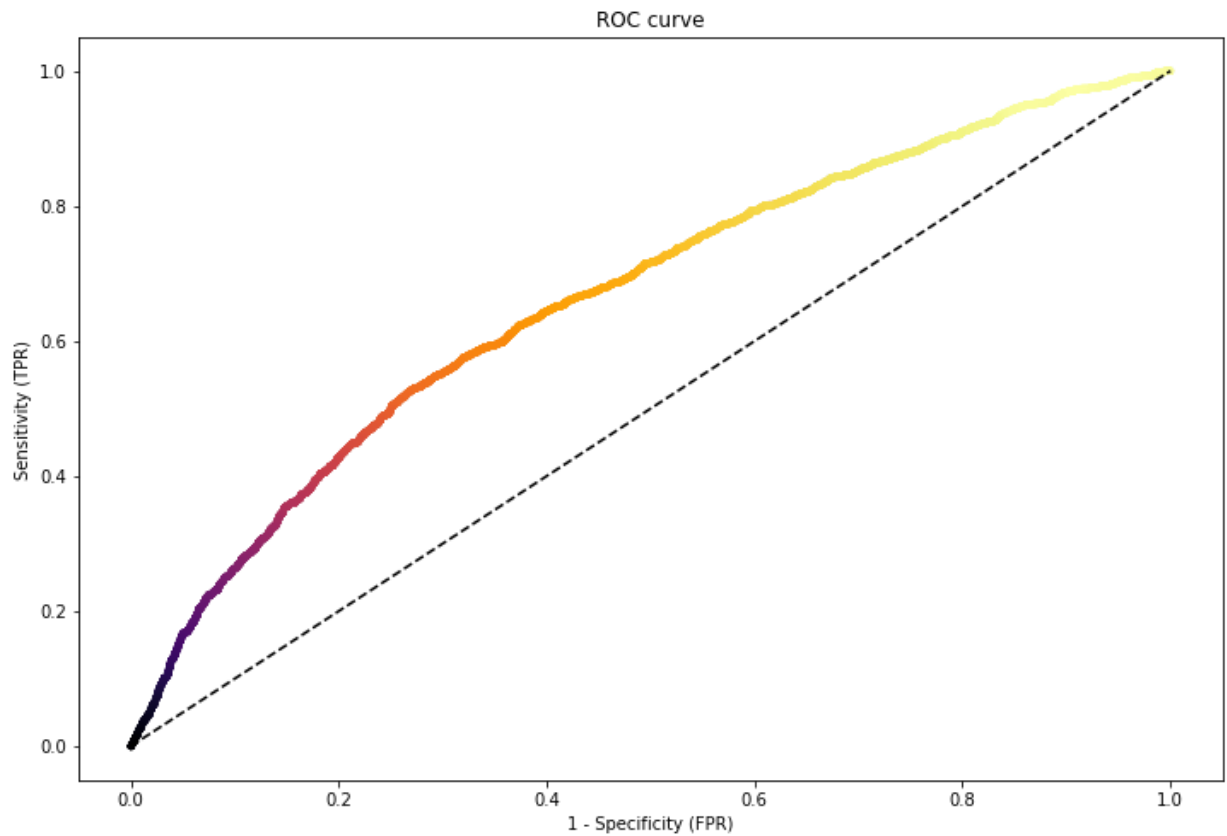
```
In [24]: fig, ax = plt.subplots()
        ax.plot(1 - spec, sens, "r-")
        ax.plot([0,1],[0,1], "k--")
        ax.set(xlabel = "1 - Specificity (FPR)",
              ylabel = "Sensitivity (TPR)",
              title = "ROC curve");
```



C.

ROC curve with negative loss coloring.

```
In [25]: from matplotlib import cm #For nice colors
fig, ax = plt.subplots()
ax.scatter(1 - spec, sens, c = -1 * loss, cmap = cm.inferno, s = 10)
ax.plot([0,1],[0,1], "k--")
ax.set(xlabel = "1 - Specificity (FPR)",
      ylabel = "Sensitivity (TPR)",
      title = "ROC curve");
```



d.

```
In [26]: min_loss_idx = np.argmin(loss)
min_loss_spec = spec[min_loss_idx]
min_loss_sens = sens[min_loss_idx]
```

```
In [36]: print(min_loss_sens,
min_loss_spec,
loss[min_loss_idx])
```

```
0.9703488372093023 0.09704545454545455 64695
```

```
In [27]: def abline(slope, intercept):
        """
        Plot a line from slope and intercept
        (source https://stackoverflow.com/a/43811762)
        """
        axes = plt.gca()
        x_vals = np.array(axes.get_xlim())
        y_vals = intercept + slope * x_vals
        plt.plot(x_vals, y_vals, '--')
```

```
In [28]: truth      = np.array(y_test, dtype = bool)

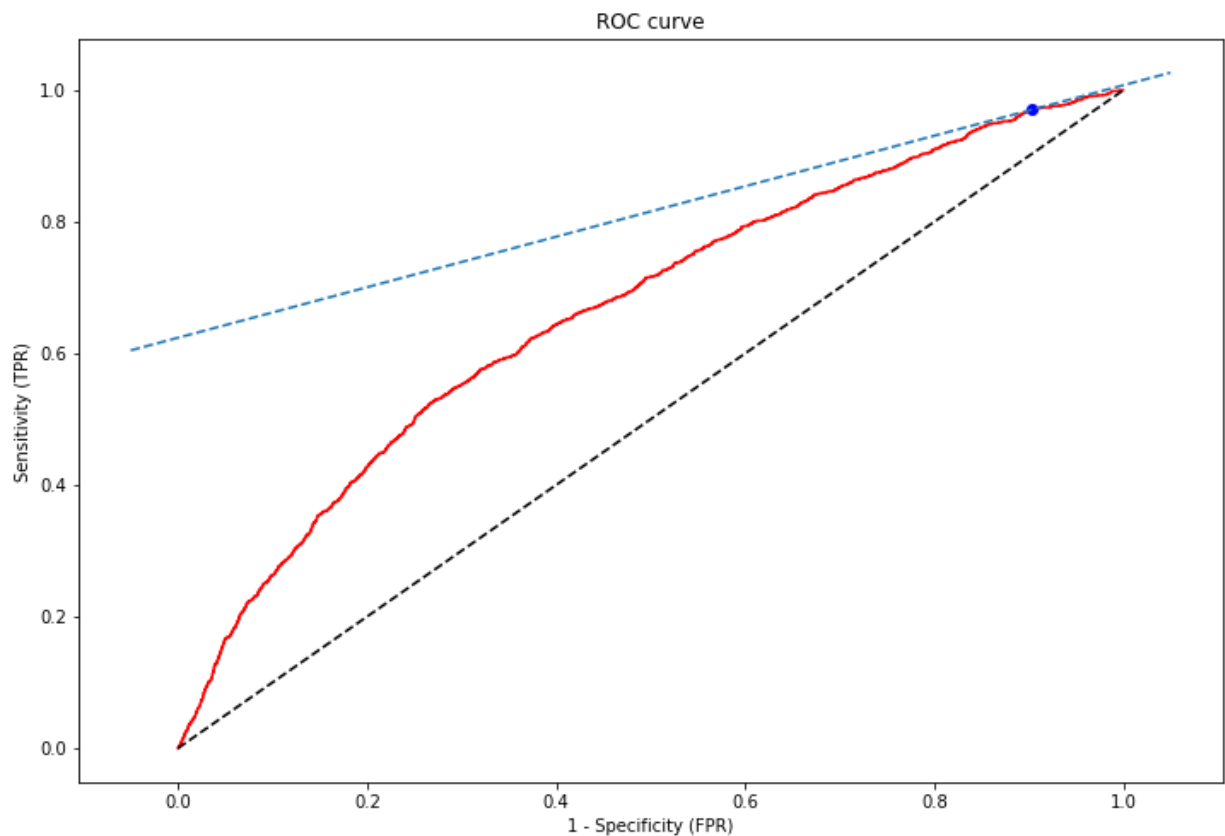
nP       = np.sum(truth)
nN       = np.sum(~truth)
loss_FP  = 5
loss_FN  = 100

slope    = nN/nP * loss_FP/loss_FN

intercept = min_loss_sens - (1 - min_loss_spec) * slope
# ^y - x * slope = intercept
```

```
In [29]: fig, ax = plt.subplots()
ax.plot(1 - spec, sens, "r-")
ax.plot([0,1],[0,1], "k--")
ax.plot(1 - min_loss_spec, min_loss_sens, "bo")
ax.set(xlabel = "1 - Specificity (FPR)",
      ylabel = "Sensitivity (TPR)",
      title = "ROC curve");

abline(slope, intercept)
```



The line with the $\frac{N}{P} \cdot \frac{L_{FP}}{L_{FN}}$ slope and intersecting the minimum loss point is to be tangential to the ROC curve.

e.

$$\begin{aligned}
\mathcal{L} &= L_{FP} \cdot FPR \cdot P + L_{FN} \cdot FNR \cdot N \\
&= L_{FP} \cdot P \cdot \int_{-\infty}^T f_1(x) dx + L_{FN} \cdot N \cdot \int_T^{\infty} f_0(x) dx \\
&= L_{FP} \cdot P \cdot \int_{-\infty}^T f_1(x) dx + L_{FN} \cdot N \cdot \left(1 - \int_{-\infty}^T f_0(x) dx\right)
\end{aligned}$$

Thus,

$$\frac{d\mathcal{L}}{dT} = L_{FP} \cdot P \cdot f_1(T) - L_{FN} \cdot N \cdot f_0(T)$$

Set equation to 0:

$$\begin{aligned}
L_{FP} \cdot P \cdot f_1(T) - L_{FN} \cdot N \cdot f_0(T) &= 0 \\
L_{FP} \cdot P \cdot f_1(T) &= L_{FN} \cdot N \cdot f_0(T) \\
L_{FP} \cdot P \cdot \frac{f_1(T)}{f_0(T)} &= L_{FN} \cdot N
\end{aligned}$$

f)

```
In [30]: # loss_FP = 5
# loss_FN = 100
log_optimal_threshold = np.log(loss_FP/loss_FN)

optimal_sens, optimal_spec, optimal_loss = eval_estimate(
    test_prob_logit > log_optimal_threshold,
    y_test, 5, 100)
```

```
In [31]: print(optimal_sens, optimal_spec, optimal_loss)

0.9598837209302326 0.11166666666666666 65530
```

```
In [32]: min(loss)
```

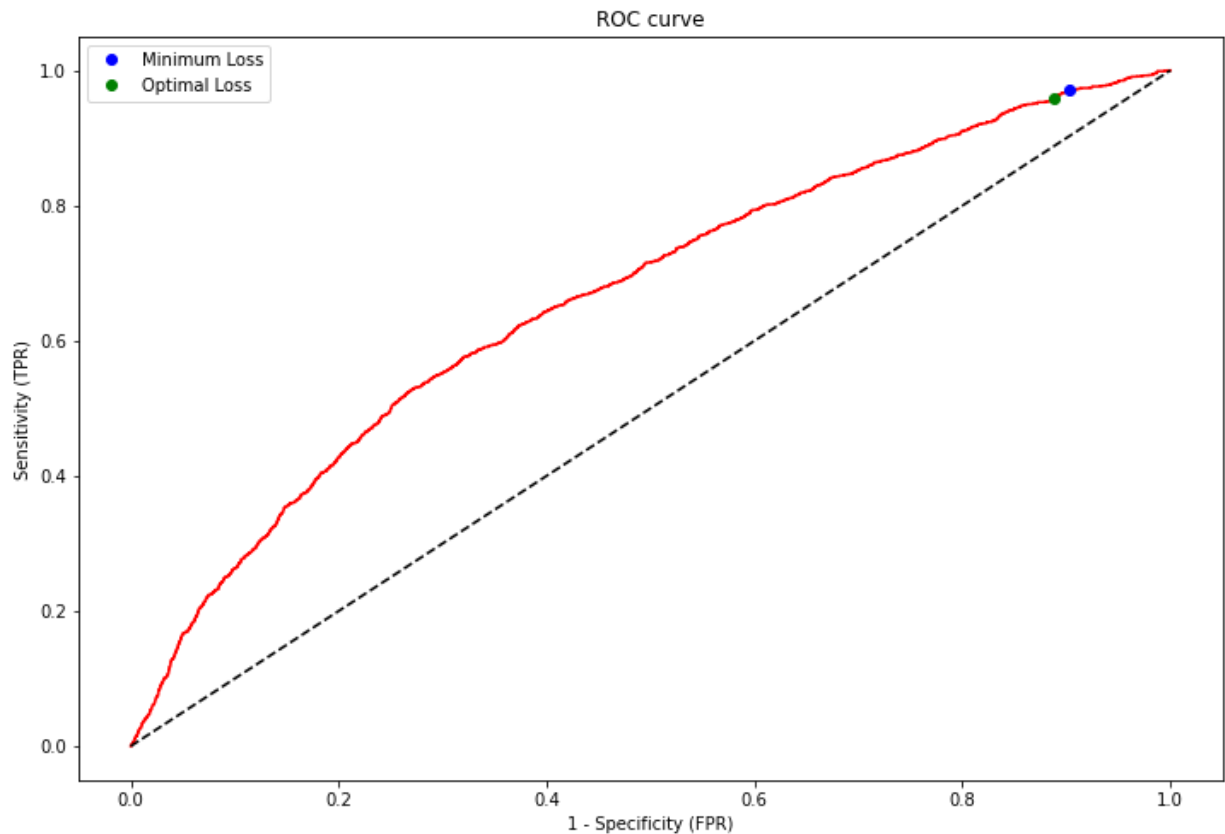
```
Out[32]: 64695
```

This loss is worse than the minimum loss we observed above. This makes sense, as our estimated probabilities are probably not the true ones, and so our theoretically optimal threshold is not actually optimal.

g.

Comparing optimal threshold vs the threshold that minimizes the empirical loss.

```
In [33]: fig, ax = plt.subplots()
ax.plot(1 - spec, sens, "r-")
ax.plot([0,1],[0,1], "k--")
ax.plot(1 - min_loss_spec, min_loss_sens, "bo", label = "Minimum Loss")
ax.plot(1 - optimal_spec, optimal_sens, "go", label = "Optimal Loss")
ax.legend()
ax.set(xlabel = "1 - Specificity (FPR)",
      ylabel = "Sensitivity (TPR)",
      title = "ROC curve");
```



In this situation we observe the mathematical optimal threshold to have higher TPR and FPR than the empirical based solution (the minimum loss). This difference may be due to the fact that the empirical minimal loss threshold is calculated with the actually points and the optimal loss is calculated by optimizing the expected loss and assuming that we have correctly calibrated probabilities.